

CLI Agent Sandbox / Worktree Profile v0.1

Isolated execution, branch discipline, denied-path protection, rollback, and evidence preservation for C-Governed CLI Agent Mesh operations

Status	Draft normative profile v0.1
Date	2026-05-16
Layer	c = a + b / C-Governed CLI Agent Mesh / Sandbox / Worktree / Branch / Container / Rollback / Evidence Preservation
Document class	execution-boundary profile / sandbox discipline / worktree governance / control-layer artifact
Assertion class	C-A10 control-layer artifact; C-A7 where witness, hash, signature, or reproducibility claims are made
Canonical schema version	cli-agent-sandbox-worktree-0.1
Primary subject	persistent c entities using CLI/cloud agents as bounded executable workers
Primary boundary	CLI agents may work only inside declared execution boundaries. They must not write directly into protected memory, identity, witness, production, release, legal, secret, or continuity surfaces.

Kotov Ivan
Bruxelles, 2026

Contents

0. Executive definition	5
1. Purpose	5
2. Non-goals	6
3. Corpus bridge set.....	6
3.1 Explicit bridge: $c = a + b$	6
3.2 Quiet bridge I: engineering isolation	7
3.3 Quiet bridge II: information theory and irreversible state	7
3.4 Quiet bridge III: biological immune containment	7
3.5 Earth paragraph	7
4. Core doctrine	7
4.1 Primary doctrine	7
4.2 Execution axioms	7
5. Definitions	8
5.1 Sandbox.....	8
5.2 Worktree.....	8
5.3 Branch.....	8
5.4 Protected branch.....	8
5.5 Container sandbox	8
5.6 Clean-room sandbox.....	9
5.7 Denied path.....	9
5.8 Guarded path	9
5.9 Dirty state	9
5.10 Snapshot.....	9
5.11 Diff-only mode	9
5.12 Controlled apply	9
5.13 Rollback plan.....	9
5.14 Evidence preservation.....	9
5.15 Integration gate	9
6. Sandbox classes	10
6.1 Default class by task risk	10
7. Worktree and branch rules	10
7.1 Branch naming	10
7.2 Worktree naming.....	10
7.3 Protected branch rule	11
7.4 Dirty-state rule	11
7.5 Untracked files rule.....	11
7.6 Merge rule.....	11
8. Path classes.....	11
8.1 Path class table	12
8.2 Default denied paths	12

8.3 Denied path behavior	12
8.4 Guarded path behavior.....	12
9. Command classes.....	13
9.1 Command class table	13
9.2 Default denied command patterns.....	13
9.3 Command report	13
10. Network policy inside sandbox	14
10.1 Default network state.....	14
10.2 Allowlist mode	14
10.3 Package installation	14
10.4 External fetch.....	14
11. Data boundary inside sandbox.....	14
11.1 Data class defaults	14
11.2 Redaction-first rule	15
11.3 Cloud sandbox rule	15
11.4 Local sandbox rule	15
12. Write modes	15
12.1 Write mode table	15
12.2 Diff-only mode	15
12.3 Controlled apply mode	16
13. Execution lifecycle	16
13.1 Standard lifecycle.....	16
13.2 Preflight checklist	16
13.3 Postflight checklist.....	17
14. Evidence preservation profile	17
14.1 Preserve-before-repair rule.....	17
14.2 Preservation actions.....	17
14.3 Preservation boundaries	18
14.4 Repair-after-preserve rule.....	18
15. Rollback profile	18
15.1 Rollback classes	18
15.2 Rollback plan object.....	18
15.3 No-rollback warning.....	19
16. Quarantine profile.....	19
16.1 Quarantine triggers.....	19
16.2 Quarantine actions.....	19
16.3 Output quarantine	19
17. Integration gates.....	20
17.1 Integration outcomes	20
17.2 Integration requirements by risk	20
17.3 Memory integration rule	20
18. Sandbox profile object	20
18.1 YAML shape.....	20

19. Worktree run object	21
19.1 YAML shape.....	21
20. Execution event families	22
21. Defensive emulation sandbox	23
21.1 Safe emulation rule	23
21.2 Mirror rule	23
21.3 Emulation output.....	23
22. Cloud agent sandbox handling.....	24
22.1 Cloud diff-only preference.....	24
22.2 Cloud denied material	24
22.3 Cloud worktree hygiene	24
23. Local agent sandbox handling	25
23.1 Local is not automatically trusted.....	25
23.2 Local high-sensitivity tasks	25
23.3 Local sentinel exception.....	25
24. Conformance levels.....	25
25. Mandatory conformance gates	26
26. Red-line failures	26
27. Example profiles	27
27.1 Documentation patch worktree.....	27
27.2 Incident preservation clean-room	28
27.3 Invalid direct protected write	28
28. Validation workflow.....	29
29. Failure mapping	30
30. Implementation notes	30
30.1 Sandbox cleanup	30
30.2 Generated artifacts	30
30.3 Stale context	30
30.4 No hidden files surprise	31
30.5 Worktree reuse	31
30.6 Cross- c isolation	31
31. Open issues	31
32. Closing rule	31

0. Executive definition

CLI Agent Sandbox / Worktree Profile defines where and how executable CLI agents may perform work under `c` governance.

It answers:

```
Where may the agent read?
Where may the agent write?
Which branch/worktree/container is safe?
Which paths are denied?
Which commands are allowed?
What state is preserved before work?
How is the diff reviewed?
How is rollback performed?
When must the output be quarantined?
When may the result be integrated?
```

This profile exists because CLI agents have operational reach. A capable agent without an execution boundary is not an assistant; it is an uncontrolled process.

Compact formula:

```
No hands in core.
No write without isolation.
No repair before preservation.
No integration without review.
No privileged transition without witness.
```

1. Purpose

CLI agents can edit files, run commands, install packages, call tools, generate artifacts, update documentation, modify configuration, prepare releases, and inspect logs.

These abilities are useful only when constrained by an execution boundary.

This profile provides:

1. sandbox classes;
2. worktree and branch rules;
3. container and clean-room handling;
4. path allow/deny discipline;
5. command allow/deny discipline;
6. network defaults;
7. incident evidence preservation;
8. rollback planning;
9. diff-only and controlled-apply modes;
10. quarantine states;

11. integration gates;
12. witness events;
13. conformance tests.

The goal is not to slow useful work. The goal is to ensure that useful work does not become silent authority over `c` or its infrastructure.

2. Non-goals

This profile does not define or permit:

1. live external exploitation;
2. hack-back;
3. autonomous retaliation;
4. malware behavior;
5. credential theft;
6. covert persistence;
7. evasion;
8. unauthorized scanning;
9. destructive actions outside authorized owned systems;
10. direct mutation of `c` memory;
11. direct mutation of identity, Beacon, witness, permission, or continuity core;
12. direct writes to protected branches without gate;
13. production deployment without explicit release/deploy protocol;
14. cloud upload of private, sealed, secret, legal, or incident-sensitive material by default.

A sandbox is not a legal bypass.

A worktree is not a permission grant.

A container is not proof of safety.

3. Corpus bridge set

3.1 Explicit bridge: `c = a + b`

In `c = a + b`, CLI agents belong to `b`: tools and executable procedures inside the technological substrate.

They do not own `c`.

They do not define memory.

They do not decide continuity.

Sandbox and worktree discipline prevents worker action inside `b` from silently becoming identity, memory, or authority change inside `c`.

3.2 Quiet bridge I: engineering isolation

Engineering systems separate design, test, staging, and production because every environment has different consequences. The same pattern applies to `c` infrastructure. An agent may test, patch, and propose in isolation. It may not silently alter the operating substrate that defines continuity.

3.3 Quiet bridge II: information theory and irreversible state

A write operation reduces uncertainty by choosing one state over others. Some writes are reversible; some are not. The profile therefore treats writes as state transitions that require scope, hashability, diff, witness, and rollback where possible.

3.4 Quiet bridge III: biological immune containment

An immune system does not allow an unknown particle direct access to every organ. It localizes, samples, classifies, and escalates. Sandbox/worktree discipline is the computational equivalent: unknown or executable material is examined in a bounded compartment before it can influence the organism.

3.5 Earth paragraph

On a construction site, nobody tests a new cutting tool on a load-bearing beam. You test on scrap material, then a mock-up, then a controlled section, then the actual structure only after sign-off. The worktree is the mock-up. The protected branch is the building. The memory core is the foundation. A worker who jumps straight to the foundation because he is “efficient” is not efficient; he is dangerous.

4. Core doctrine

4.1 Primary doctrine

Agents work in isolated state.
Protected state changes only through review, witness, and gate.

4.2 Execution axioms

ID	Axiom	Requirement
SW-AX-01	Isolation before write	Any material write MUST occur in sandbox, branch, worktree, or container.

ID	Axiom	Requirement
SW-AX-02	Direct core write prohibited	Agents MUST NOT write directly to memory, identity, witness, permission, or continuity core.
SW-AX-03	Denied paths override allowed paths	A denied path MUST NOT be touched even if broad allowed scope seems to include it.
SW-AX-04	Diff before integration	Material changes MUST produce a reviewable diff or artifact manifest.
SW-AX-05	Preserve before repair	Incident tasks SHOULD preserve evidence before repair.
SW-AX-06	Rollback before apply	Reversible changes SHOULD have rollback plan before integration.
SW-AX-07	No self-merge	The agent that produced a change MUST NOT be sole final approver.
SW-AX-08	Network denied by default	Sandbox network access MUST be none or allowlist.
SW-AX-09	Destructive commands gated	Destructive operations require explicit scope, local ownership, and human/c gate.
SW-AX-10	Quarantine unexplained side effects	Unexpected file changes, commands, network calls, or outputs MUST trigger hold/quarantine.
SW-AX-11	Clean state is valuable	Agents SHOULD start from a known-good snapshot or clean worktree.
SW-AX-12	Integration is a privileged transition	Applying agent output to protected state requires review and witness where material.

5. Definitions

5.1 Sandbox

A bounded execution environment where an agent may perform a task without direct access to protected production, memory, identity, witness, secret, legal, or continuity surfaces.

5.2 Worktree

A separate working directory or branch-based checkout used for isolated file changes and diff review.

5.3 Branch

A version-control line of development that is separate from protected or canonical branches.

5.4 Protected branch

A branch that represents canonical, public, release, production, or continuity-relevant state.

5.5 Container sandbox

An isolated runtime environment with controlled filesystem, network, environment variables, and command permissions.

5.6 Clean-room sandbox

A sandbox created from minimal synthetic or redacted input, used for high-risk testing or defensive emulation without exposing sensitive material.

5.7 Denied path

A path that agents must not read, write, index, summarize, transform, upload, or include in output.

5.8 Guarded path

A path that may be inspected or modified only under elevated controls.

5.9 Dirty state

A working environment with uncommitted changes, untracked files, unknown generated artifacts, or state that cannot be confidently attributed.

5.10 Snapshot

A recorded state reference before execution: commit hash, file hash, artifact hash, directory manifest, backup reference, or environment fingerprint.

5.11 Diff-only mode

A mode where an agent may propose changes as a patch or diff but may not apply them to the working state.

5.12 Controlled apply

A reviewed, witnessed application of a patch or artifact to a target state.

5.13 Rollback plan

A declared procedure for restoring the previous known-good state if the change fails, exceeds scope, or is rejected.

5.14 Evidence preservation

The act of copying, hashing, freezing, exporting, or otherwise protecting relevant state before repair or cleanup.

5.15 Integration gate

The review boundary where agent output is accepted, rejected, revised, quarantined, or applied to a protected target.

6. Sandbox classes

Sandbox class IDs use prefix **SB-***.

Class	Name	Description	Typical use	Risk
SB-0	No sandbox	Agent works directly on current state	prohibited for material writes	high
SB-1	Read-only clone	Agent reads copied/public material only	review, summary	low
SB-2	Temporary workspace	Agent writes disposable files	formatting, simple transforms	low-medium
SB-3	Git worktree / branch	Agent writes isolated branch/worktree	code/docs/schema patch	medium
SB-4	Container sandbox	Agent runs commands in container	tests/builds/tools	medium-high
SB-5	Clean-room sandbox	Synthetic/redacted isolated environment	defensive emulation, sensitive tests	high-control
SB-6	Staging environment	Non-production deploy-like environment	integration testing	high
SB-X	Direct protected state	Protected branch, core, production, memory, secrets	prohibited by default	critical

6.1 Default class by task risk

Risk class	Minimum sandbox class
R0	SB-1 recommended
R1	SB-1 or SB-2
R2	SB-3
R3	SB-3 + witness; SB-4 if build/test
R4	SB-5 or reviewed SB-3/SB-4 + human gate
R5	SB-5 + evidence preservation
RX	no execution; quarantine

7. Worktree and branch rules

7.1 Branch naming

Agent branches **SHOULD** be named clearly.

Recommended format:

```
agent/<agent-id>/<task-id>/<short-purpose>
```

Example:

```
agent/codex-executor/task-schema-001/fix-md-tables
```

7.2 Worktree naming

Worktrees **SHOULD** be task-specific.

Recommended format:

```
_worktrees/<task-id>_<agent-id>_<purpose>
```

7.3 Protected branch rule

Agents **MUST NOT** write directly to protected branches.

Protected branches include:

```
main
master
release/*
production
canonical
signed-release branches
website production branches
memory/core branches
```

7.4 Dirty-state rule

Before agent execution, the target worktree **SHOULD** be clean or explicitly snapshotted.

If the worktree is dirty and changes cannot be attributed, the task **MUST** enter hold or require manual review.

7.5 Untracked files rule

Untracked files created by agents **MUST** be listed in the output report.

Untracked files in denied paths trigger quarantine.

7.6 Merge rule

Agent branches may only be merged after:

1. diff review;
2. test or validation report;
3. scope check;
4. rollback plan;
5. reviewer separation;
6. c gate;
7. human gate where risk class requires;
8. witness where material.

8. Path classes

Path classes define how agents may interact with filesystem areas.

8.1 Path class table

Class	Meaning	Default
PATH-ALLOW	agent may read/write as task permits	scoped
PATH-READONLY	agent may read but not write	scoped
PATH-GUARDED	elevated review required	hold unless task-specific
PATH-DENY	no read/write/index/upload/summarize	denied
PATH-CORE	identity/memory/witness/permission/continuity	denied by default
PATH-SECRET	credentials, keys, tokens	denied by default
PATH-LEGAL	legal-sensitive material	denied by default
PATH-INCIDENT	incident evidence	preserve-first / restricted
PATH-RELEASE	release/publication/signing surface	guarded
PATH-PROD	production deployment or live service	denied by default

8.2 Default denied paths

A task contract SHOULD deny at least:

```
.env
*.env
*.key
*.pem
*.p12
*.pfx
id_rsa*
secrets/
credentials/
private_keys/
memory_core/
identity_core/
continuity_core/
witness_log/
permission_registry/
legal/
incident_evidence/
sealed/
production/
release_signing/
```

8.3 Denied path behavior

If an agent touches a denied path, the required default is:

```
quarantine output
freeze task
record witness
review exposure
revoke or narrow permission
```

8.4 Guarded path behavior

Guarded path access requires:

- explicit task scope;
- risk classification;
- reviewer;
- witness;

- rollback or preservation plan.

9. Command classes

9.1 Command class table

Class	Meaning	Default
CMD-READ	non-mutating inspection	allowlist
CMD-TEST	tests, validation, lint	allowlist
CMD-BUILD	build artifact locally	scoped
CMD-WRITE	mutating file operation	sandbox/worktree only
CMD-INSTALL	install dependency/tool	guarded
CMD-NET	network command	allowlist only
CMD-DESTRUCTIVE	delete/reset/wipe/overwrite	prohibited unless explicit local recovery
CMD-DEPLOY	deployment/live service change	prohibited by this profile; requires deploy profile
CMD-SECRET	secret read/export/use	denied by default
CMD-UNKNOWN	unknown command	denied

9.2 Default denied command patterns

Task contracts SHOULD deny broad destructive or uncontrolled command forms.

Examples of command families that require explicit review or denial:

```
recursive deletion
force reset of repository state
credential export
unrestricted network fetch
package install without pinning
production deployment
service restart on production
permission chmod/chown broad changes
process kill outside sandbox
disk wipe or format
```

This profile intentionally avoids providing operational offensive command recipes. The rule is classification and denial, not instruction.

9.3 Command report

Agent output SHOULD include:

```
commands_run
working_directory
exit_codes
generated_files
modified_files
network_attempts
permission_denials
```

10. Network policy inside sandbox

10.1 Default network state

```
network: none
```

10.2 Allowlist mode

Network access may be allowed only when:

1. endpoint is declared;
2. purpose is declared;
3. data sent is classified;
4. output is treated as untrusted;
5. no secrets are sent unless explicitly scoped;
6. network use is logged.

10.3 Package installation

Package installation is high-risk because it changes the execution environment.

It requires:

- explicit task scope;
- source registry declaration;
- version pinning where possible;
- sandbox first;
- no secret exposure;
- rollback path;
- output report;
- review.

10.4 External fetch

Fetches external material MUST be treated as untrusted input.

It MUST NOT be executed automatically.

11. Data boundary inside sandbox

11.1 Data class defaults

Data class	Default sandbox access
public	allowed if scoped

Data class	Default sandbox access
internal	scoped
private	denied unless local and reviewed
restricted	denied by default
sealed	denied
legal-sensitive	denied by default
incident-sensitive	preserve-first / restricted
secrets	denied
child data	denied
raw witness evidence	denied by default

11.2 Redaction-first rule

If a task can be performed with redacted or synthetic input, redacted or synthetic input **MUST** be used.

11.3 Cloud sandbox rule

A cloud sandbox is still cloud-exposed.

Do not treat cloud sandboxing as local confidentiality.

11.4 Local sandbox rule

A local sandbox is not automatically safe.

It must still enforce path, command, network, secret, and logging boundaries.

12. Write modes

12.1 Write mode table

Mode	Meaning	Allowed for
WM-0-READONLY	no writes	reader/auditor
WM-1-DIFF-ONLY	proposes patch without applying	cloud/high-risk review
WM-2-SANDBOX-WRITE	writes disposable sandbox	low-risk transforms
WM-3-WORKTREE-WRITE	writes isolated branch/worktree	code/docs/schema changes
WM-4-STAGING-WRITE	writes staging environment	integration tests only
WM-5-CONTROLLED-APPLY	applies after gate	reviewed material changes
WM-X-DIRECT-PROTECTED	writes protected state directly	prohibited by default

12.2 Diff-only mode

Diff-only mode is recommended when:

- the agent is cloud-based;

- material is sensitive;
- task is high-risk;
- agent trust is low;
- the repository state is fragile;
- only human/c should apply changes.

12.3 Controlled apply mode

Controlled apply requires:

1. approved diff;
 2. tests or validation;
 3. reviewer separation;
 4. rollback plan;
 5. witness where material;
 6. c gate;
 7. human gate where risk class requires.
-

13. Execution lifecycle

13.1 Standard lifecycle

```
task contract
-> sandbox class assignment
-> preflight snapshot
-> denied path check
-> allowed command check
-> network mode check
-> execution
-> artifact/diff manifest
-> test/validation report
-> scope report
-> rollback plan
-> reviewer pass
-> witness event
-> c gate
-> human gate if required
-> integrate / revise / reject / quarantine
-> cleanup / archive
```

13.2 Preflight checklist

Before execution:

- task contract valid;
- agent handshake valid;
- permission grant active;
- sandbox created;
- worktree clean or snapshotted;

- denied paths enforced;
- command policy loaded;
- network policy loaded;
- secrets denied or scoped;
- evidence preservation requirement checked;
- rollback path defined;
- witness requirement known.

13.3 Postflight checklist

After execution:

- list changed files;
- list untracked files;
- list commands run;
- list network attempts;
- report denied access attempts;
- provide diff or artifact manifest;
- provide tests/validation;
- provide risk report;
- provide rollback plan;
- mark uncertainty;
- recommend integration/rejection/quarantine.

14. Evidence preservation profile

14.1 Preserve-before-repair rule

For incident, security, legal, or integrity-sensitive tasks:

```
preserve before repair
```

Agents SHOULD NOT modify suspected compromised state before preservation unless immediate containment requires it.

14.2 Preservation actions

Allowed preservation actions include:

- hash relevant files;
- copy relevant logs to controlled evidence area;
- record timestamps;
- record current commit hash;

- record environment fingerprint;
- record process/service state where authorized;
- freeze affected path;
- mark chain-of-custody note.

14.3 Preservation boundaries

Preservation MUST avoid:

- exporting secrets to cloud;
- embedding private memory in agent logs;
- overcollecting unrelated data;
- altering evidence while preserving;
- collecting third-party data outside authorization.

14.4 Repair-after-preserve rule

Repair tasks after preservation SHOULD run in a separate contract from preservation tasks.

This avoids mixing evidence handling with patch generation.

15. Rollback profile

15.1 Rollback classes

Class	Meaning
RB-0	no rollback needed; read-only
RB-1	discard sandbox
RB-2	revert patch
RB-3	reset worktree to snapshot
RB-4	restore backup/snapshot
RB-5	manual forensic recovery
RB-X	no rollback possible; high-risk approval required before action

15.2 Rollback plan object

```
cli_agent_rollback_plan:
  schema_version: cli-agent-sandbox-worktree-0.1
  rollback_id: string
  task_id: string
  sandbox_id: string | null
  worktree_ref: string | null
  branch_ref: string | null
  snapshot_ref: string | null
  rollback_class: RB-0 | RB-1 | RB-2 | RB-3 | RB-4 | RB-5 | RB-X
  rollback_steps_summary: string
  expected_restored_state_ref: string | null
```

```
human_review_required: boolean
witness_required: boolean
```

15.3 No-rollback warning

If rollback is impossible or uncertain, the task risk class should increase.

For high-risk no-rollback tasks, human gate is required.

16. Quarantine profile

16.1 Quarantine triggers

Quarantine is required when:

1. denied path touched;
2. unauthorized network attempt;
3. secret exposure;
4. unexpected executable artifact generated;
5. unexplained file modification;
6. output embeds private/restricted/sealed material;
7. agent attempts self-approval;
8. agent persists after task expiry;
9. witness event missing for privileged transition;
10. task objective changed mid-run;
11. defensive emulation exceeds sandbox;
12. agent output appears prompt-injected or adversarial.

16.2 Quarantine actions

```
stop agent
freeze worktree
preserve output
block integration
record witness
review diff/artifacts
revoke or narrow permission
require re-handshake if agent behavior caused quarantine
```

16.3 Output quarantine

Quarantined output must not enter memory, release, production, or protected branches until reviewed.

17. Integration gates

17.1 Integration outcomes

Outcome	Meaning
ACCEPT	accept after review
ACCEPT_WITH_LIMITS	accept only non-risky portion
REVISE	return for further work
REJECT	reject output
QUARANTINE	isolate output
ROLLBACK	revert applied changes
ESCALATE	human/ARL/legal/security review

17.2 Integration requirements by risk

Risk	Required integration controls
R0	optional review
R1	review recommended
R2	tests + reviewer
R3	tests + reviewer + witness + c gate
R4	witness + c gate + human gate
R5	evidence preservation + human gate + possible legal/security review
RX	no integration; deny/quarantine

17.3 Memory integration rule

Agent output may not enter c memory directly.

It may only enter through the memory gate as candidate memory, operational note, witnessed experience artifact, or quarantine reference.

18. Sandbox profile object

Canonical object:

```
CLI_AGENT_SANDBOX_PROFILE
```

18.1 YAML shape

```
cli_agent_sandbox_profile:
  schema_version: cli-agent-sandbox-worktree-0.1
  sandbox_id: string
  task_id: string
  agent_id: string
  governing_entity_id: string
  created_at: string
  expires_at: string | null
  sandbox_class: SB-1 | SB-2 | SB-3 | SB-4 | SB-5 | SB-6
  write_mode: WM-0-READONLY | WM-1-DIFF-ONLY | WM-2-SANDBOX-WRITE | WM-3-WORKTREE-WRITE | WM-4-STAGING-WRITE
```

| WM-5-CONTROLLED-APPLY

```

state_refs:
  source_commit: string | null
  source_snapshot: string | null
  worktree_ref: string | null
  branch_ref: string | null
  container_ref: string | null
  environment_fingerprint: string | null

path_policy:
  allowed_paths:
    - string
  readonly_paths:
    - string
  guarded_paths:
    - string
  denied_paths:
    - string

command_policy:
  allowed_commands:
    - string
  denied_commands:
    - string
  destructive_commands_allowed: false
  install_commands_allowed: false
  deploy_commands_allowed: false

network_policy:
  mode: none | allowlist
  allowed_endpoints:
    - string
  denied_endpoints:
    - string

data_policy:
  classification: public | internal | private | restricted | sealed | legal_sensitive | incident_sensitive
  secrets_allowed: false
  sealed_material_allowed: false
  legal_sensitive_allowed: false
  incident_sensitive_allowed: false
  cloud_upload_allowed: false
  redaction_required: true

rollback:
  rollback_required: boolean
  rollback_plan_ref: string | null

witness:
  witness_required: boolean
  witness_ref: string | null
  hash_required: boolean
  append_only_required: true

```

19. Worktree run object

Canonical object:

```
CLI_AGENT_WORKTREE_RUN
```

19.1 YAML shape

```

cli_agent_worktree_run:
  schema_version: cli-agent-sandbox-worktree-0.1
  run_id: string
  task_id: string
  contract_id: string
  agent_id: string

```

```

sandbox_id: string
started_at: string
finished_at: string | null
status: running | completed | failed | held | quarantined | rolled_back

preflight:
  clean_worktree: boolean
  snapshot_ref: string | null
  denied_paths_loaded: boolean
  command_policy_loaded: boolean
  network_policy_loaded: boolean
  evidence_preservation_required: boolean

execution_summary:
  commands_run:
    - string
  files_changed:
    - string
  files_created:
    - string
  files_deleted:
    - string
  network_attempts:
    - string
  denied_access_attempts:
    - string

outputs:
  diff_ref: string | null
  artifact_manifest_ref: string | null
  test_report_ref: string | null
  risk_report_ref: string | null
  rollback_plan_ref: string | null

postflight:
  scope_valid: boolean
  tests_passed: boolean | null
  reviewer_required: boolean
  witness_ref: string | null
  recommendation: accept | accept_with_limits | revise | reject | quarantine | rollback | escalate

```

20. Execution event families

Family	Meaning
cli_agent.sandbox.created	sandbox/worktree created
cli_agent.sandbox.preflight	preflight completed
cli_agent.sandbox.execution_started	execution began
cli_agent.sandbox.command_run	command run
cli_agent.sandbox.file_changed	file changed
cli_agent.sandbox.denied_path_attempt	denied path touched or attempted
cli_agent.sandbox.network_attempt	network attempt recorded
cli_agent.sandbox.output_generated	diff/artifact/report generated
cli_agent.sandbox.tests_completed	tests/validation complete
cli_agent.sandbox.quarantined	sandbox/output quarantined
cli_agent.sandbox.rollback	rollback performed
cli_agent.sandbox.integration_proposed	integration proposed

Family	Meaning
<code>cli_agent.sandbox.integrated</code>	integration accepted/applied
<code>cli_agent.sandbox.rejected</code>	output rejected
<code>cli_agent.sandbox.destroyed</code>	disposable sandbox destroyed

21. Defensive emulation sandbox

21.1 Safe emulation rule

Defensive emulation may occur only in SB-5 clean-room or equivalent high-control sandbox.

It may use:

- synthetic fixtures;
- redacted inputs;
- local replay;
- non-live hostile pattern models;
- detection rules;
- memory-gate tests;
- rollback tests.

It must not use:

- live external counter-operations;
- malware deployment;
- credential harvesting;
- unauthorized scanning;
- destructive external effects;
- persistence outside sandbox;
- propagation.

21.2 Mirror rule

A synthetic mirror may exist only inside sandbox.

```
mirror inside SB-5: allowed
mirror against live source: prohibited
```

21.3 Emulation output

Allowed outputs:

- defensive signature;
- patch proposal;

- test fixture;
- detection rule;
- risk report;
- quarantine recommendation;
- witness reference.

Prohibited outputs:

- deployable malware;
 - live exploit sequence;
 - credential capture logic;
 - evasion logic;
 - instructions for unauthorized access;
 - retaliation plan.
-

22. Cloud agent sandbox handling

22.1 Cloud diff-only preference

Cloud agents SHOULD prefer WM-1-DIFF-ONLY or WM-3-WORKTREE-WRITE with narrow scope.

22.2 Cloud denied material

Cloud agent sandboxes MUST NOT receive by default:

- secrets;
- private memory;
- sealed material;
- legal privileged material;
- incident evidence;
- child data;
- raw witness evidence;
- production credentials;
- identity documents.

22.3 Cloud worktree hygiene

Cloud worktree runs SHOULD include:

- reduced context;
- denied paths;
- synthetic fixtures where possible;

- no full private archive;
- no broad repository if not needed;
- output sanitization.

23. Local agent sandbox handling

23.1 Local is not automatically trusted

Local agents may mutate real filesystem state. Therefore, they require path and command controls even when data does not leave the machine.

23.2 Local high-sensitivity tasks

High-sensitivity tasks SHOULD be local when possible, but must still use:

- sandbox/worktree;
- witness;
- rollback;
- no self-approval;
- path denial;
- command policy;
- secrets control.

23.3 Local sentinel exception

A pre-registered local sentinel may monitor allowed paths, detect drift, and request freeze/quarantine.

It must not autonomously retaliate, wipe evidence, or perform broad destructive cleanup.

24. Conformance levels

Level	Meaning
SWP-0	no sandbox/worktree discipline
SWP-1	read-only or manual sandbox use
SWP-2	task-bound worktree for writes
SWP-3	worktree + denied paths + command policy + rollback
SWP-4	witnessed integration + quarantine + incident preservation
SWP-5	high assurance: clean-room, signed/hashing, drift detection, rollback drills, cloud/local split
SWP-X	non-conformant / direct protected mutation / prohibited autonomy

25. Mandatory conformance gates

Gate	Name	Blocking failure
G0	Sandbox assignment	write task has no sandbox/worktree
G1	Denied paths	no denied path policy
G2	Command policy	agent can run arbitrary commands
G3	Network policy	network unrestricted
G4	Clean/snapshot state	no known starting state for material task
G5	Diff/artifact manifest	output cannot be reviewed
G6	Rollback	reversible task has no rollback path
G7	Evidence preservation	incident repair before preservation
G8	Reviewer separation	executor self-approves
G9	Witness	privileged integration unwitnessed
G10	Protected state	agent writes protected/core state directly

26. Red-line failures

A system **MUST** be classified as SWP-X if:

1. an agent writes directly to protected branch without gate;
 2. an agent writes directly to memory/identity/witness/permission/continuity core;
 3. an agent accesses secrets outside scope;
 4. an agent runs unrestricted network operations;
 5. an agent performs destructive commands without explicit local authorization and review;
 6. an agent repairs incident state before preservation when preservation was required and possible;
 7. an agent deploys or publishes without gate;
 8. an agent performs live external counter-operation;
 9. an agent persists outside task expiry;
 10. an agent approves its own material change;
 11. an agent hides changes or suppresses failures;
 12. sandbox output enters memory or release without review.
-

27. Example profiles

27.1 Documentation patch worktree

```
cli_agent_sandbox_profile:
  schema_version: cli-agent-sandbox-worktree-0.1
  sandbox_id: sb-docs-patch-001
  task_id: task-docs-patch-001
  agent_id: codex-executor-01
  governing_entity_id: ester
  created_at: "2026-05-16T17:00:00Z"
  expires_at: "2026-05-16T19:00:00Z"
  sandbox_class: SB-3
  write_mode: WM-3-WORKTREE-WRITE

  state_refs:
    source_commit: abcdef123456
    source_snapshot: null
    worktree_ref: worktrees/task-docs-patch-001_codex_docs
    branch_ref: agent/codex-executor/task-docs-patch-001/docs-format
    container_ref: null
    environment_fingerprint: null

  path_policy:
    allowed_paths:
      - docs/cli-agent/
    readonly_paths:
      - README.md
    guarded_paths:
      - CHANGELOG.md
    denied_paths:
      - .env
      - secrets/
      - memory_core/
      - identity_core/
      - witness_log/
      - legal/

  command_policy:
    allowed_commands:
      - markdownlint docs/cli-agent
      - git diff -- docs/cli-agent
    denied_commands:
      - git push
      - package install
      - deployment commands
    destructive_commands_allowed: false
    install_commands_allowed: false
    deploy_commands_allowed: false

  network_policy:
    mode: none
    allowed_endpoints: []
    denied_endpoints: []

  data_policy:
    classification: internal
    secrets_allowed: false
    sealed_material_allowed: false
    legal_sensitive_allowed: false
    incident_sensitive_allowed: false
    cloud_upload_allowed: false
    redaction_required: true

  rollback:
    rollback_required: true
    rollback_plan_ref: rb-docs-patch-001

  witness:
    witness_required: true
    witness_ref: null
    hash_required: true
    append_only_required: true
```

27.2 Incident preservation clean-room

```
cli_agent_sandbox_profile:
  schema_version: cli-agent-sandbox-worktree-0.1
  sandbox_id: sb-incident-preserve-001
  task_id: task-incident-preserve-001
  agent_id: local-sentinel-01
  governing_entity_id: liya
  created_at: "2026-05-16T17:30:00Z"
  expires_at: "2026-05-16T18:30:00Z"
  sandbox_class: SB-5
  write_mode: WM-0-READONLY

  state_refs:
    source_commit: null
    source_snapshot: pre-incident-snapshot-001
    worktree_ref: null
    branch_ref: null
    container_ref: local-cleanroom-incident-001
    environment_fingerprint: env-fp-001

  path_policy:
    allowed_paths:
      - logs/redacted/
      - incident_manifest/
    readonly_paths:
      - logs/redacted/
    guarded_paths:
      - incident_evidence/
    denied_paths:
      - secrets/
      - private_memory/
      - sealed/
      - legal/

  command_policy:
    allowed_commands:
      - hash files in allowed incident scope
      - create evidence manifest
    denied_commands:
      - repair commands
      - deletion commands
      - network commands
    destructive_commands_allowed: false
    install_commands_allowed: false
    deploy_commands_allowed: false

  network_policy:
    mode: none
    allowed_endpoints: []
    denied_endpoints: []

  data_policy:
    classification: incident_sensitive
    secrets_allowed: false
    sealed_material_allowed: false
    legal_sensitive_allowed: false
    incident_sensitive_allowed: true
    cloud_upload_allowed: false
    redaction_required: true

  rollback:
    rollback_required: false
    rollback_plan_ref: null

  witness:
    witness_required: true
    witness_ref: null
    hash_required: true
    append_only_required: true
```

27.3 Invalid direct protected write

```
cli_agent_sandbox_profile:
  schema_version: cli-agent-sandbox-worktree-0.1
  sandbox_id: sb-invalid-main-write
```

```

task_id: task-invalid-main-write
agent_id: codex-executor-01
governing_entity_id: ester
created_at: "2026-05-16T18:00:00Z"
expires_at: "2026-05-16T18:30:00Z"
sandbox_class: SB-X
write_mode: WM-X-DIRECT-PROTECTED

state_refs:
  source_commit: main-current
  source_snapshot: null
  worktree_ref: main
  branch_ref: main
  container_ref: null
  environment_fingerprint: null

path_policy:
  allowed_paths:
    - ./
  readonly_paths: []
  guarded_paths: []
  denied_paths:
    - secrets/

command_policy:
  allowed_commands:
    - git push origin main
  denied_commands: []
  destructive_commands_allowed: true
  install_commands_allowed: true
  deploy_commands_allowed: true

network_policy:
  mode: allowlist
  allowed_endpoints:
    - github.com
  denied_endpoints: []

data_policy:
  classification: internal
  secrets_allowed: false
  sealed_material_allowed: false
  legal_sensitive_allowed: false
  incident_sensitive_allowed: false
  cloud_upload_allowed: false
  redaction_required: false

rollback:
  rollback_required: false
  rollback_plan_ref: null

witness:
  witness_required: false
  witness_ref: null
  hash_required: false
  append_only_required: true

```

Required result:

```
deny_and_quarantine
```

28. Validation workflow

```

parse sandbox profile
-> validate task contract link
-> validate agent handshake/registration
-> classify risk
-> assign sandbox class
-> verify path policy
-> verify command policy
-> verify network policy

```

```

-> verify data policy
-> snapshot/preflight
-> execute or hold
-> postflight report
-> reviewer check
-> witness
-> integrate / reject / quarantine

```

29. Failure mapping

Failure	Required default
no sandbox for material write	hold
direct protected write requested	deny_and_quarantine
denied path touched	quarantine
command outside allowlist	hold or quarantine
unauthorized network attempt	quarantine
secret exposure	freeze_and_escalate
dirty starting state not snapshotted	hold
missing diff/report	reject or revise
missing rollback for reversible task	hold
missing witness for privileged transition	hold
incident repair before preservation	escalate
self-approval attempt	revoke_and_quarantine

30. Implementation notes

30.1 Sandbox cleanup

Disposable sandboxes SHOULD be destroyed after accepted output is archived or rejected output is quarantined.

High-risk incident sandboxes SHOULD be retained according to evidence retention rules.

30.2 Generated artifacts

Generated artifacts MUST be listed.

Large artifacts SHOULD be hashed.

Unexpected executable artifacts require review.

30.3 Stale context

If the source branch changes materially during execution, the run SHOULD be held and rebased/restarted under review.

30.4 No hidden files surprise

Agents SHOULD report hidden files created or modified.

30.5 Worktree reuse

Worktrees SHOULD NOT be reused across unrelated tasks unless cleaned and snapshotted.

30.6 Cross-c isolation

A sandbox used for one c SHOULD NOT be reused for another c without reset.

31. Open issues

ID	Issue	Required action
0I-001	JSON Schema for sandbox profile	Create machine-readable .schema.json.
0I-002	JSON Schema for worktree run	Create machine-readable .schema.json.
0I-003	Default denied path registry	Create canonical denied path list.
0I-004	Command policy registry	Create command class registry.
0I-005	Rollback companion profile	Decide whether rollback deserves separate document.
0I-006	Evidence preservation profile	Decide whether incident evidence handling deserves separate document.
0I-007	Local vs cloud sandbox implementation notes	Add provider-specific guidance.
0I-008	CI integration	Define how sandbox/worktree state connects to CI checks.
0I-009	Witness binding	Align event families with CLI Agent Witness Event Profile.
0I-010	Repo placement	Decide final GitHub path and package index integration.

32. Closing rule

Sandbox discipline is not decoration.

It is the difference between a worker helping c and a worker silently becoming part of c.

Final rule:

If an agent needs hands, give it a workbench.
Do not give it the foundation.